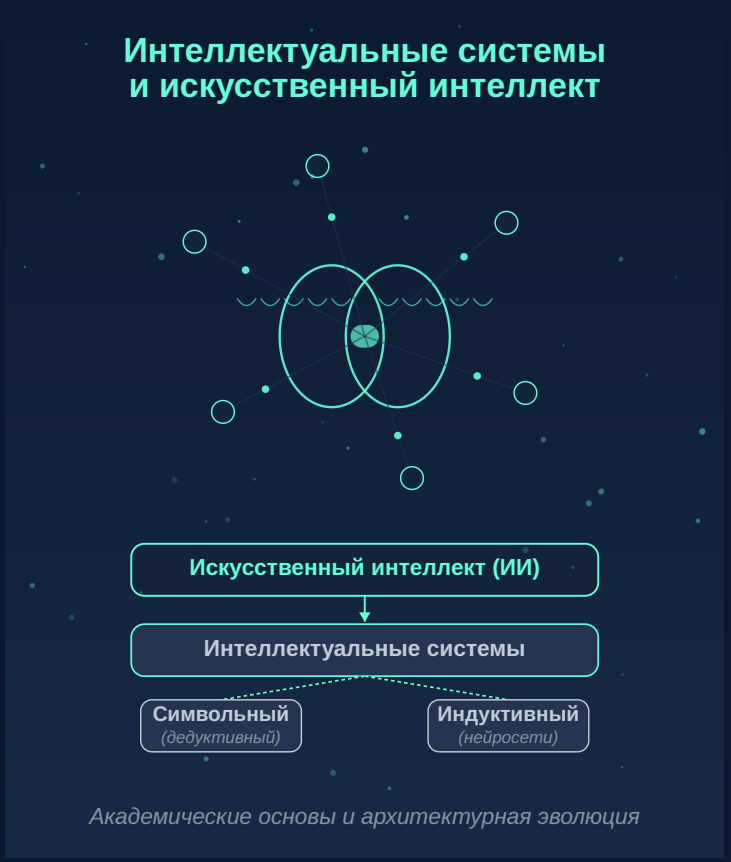
Интеллектуальные системы и искусственный интеллект (1)

Искусственный интеллект (ИИ) — свойство интеллектуальных систем решать задачи, требующие человеческого интеллекта (обучение, рассуждение, восприятие, принятие решений) в условиях неопределенности.

Интеллектуальная система — программно-аппаратный комплекс, реализующий функции интеллектуального поведения (анализ, синтез, обобщение) при решении задач в условиях неполноты или противоречивости исходной информации.



План лекции: 6 ключевых направлений (2)

1. **Компьютерное зрение** — детекция, сегментация, генерация
2. **Обработка языка** — трансформеры, языковые модели
3. **Обучение с подкреплением** — робототехника, оптимизация
4. **Генеративные модели** — GAN, диффузия, автоэнкодеры
5. **Эволюция архитектур** — от перцептрона до Mamba
6. **Будущее ИИ** — эффективность, интерпретируемость, этика

Шесть фундаментальных направлений современного ИИ

Основные определения и иерархия (3)

Иерархия понятий:
- **Искусственный интеллект** — общая концепция
- **Интеллектуальные системы** — конкретные реализации
- **Подходы:** символьный (дедуктивный) и машинное обучение (индуктивный)

Спектр систем по степени интеллектуальности (4)

Тип системы	Основа	Адаптация	Пример
Автоматическая	Жёсткие алгоритмы	Нет	Термостат
Автоматизированная	Человек + инструмент	Человеком	CAD
Интеллектуализированная	Эвристики + правила	Ограниченная	Экспертные системы
Интеллектуальная	Моделирование интеллекта	Автономная	Нейросети

Диаграмма

ıектр систем автоматические а...

Критерий интеллектуальности: способность к автономному обучению и адаптации.

Дедуктивный подход: экспертные системы (5)

Экспертная система — программа, использующая базу знаний и логический вывод для решения задач на уровне эксперта.

Архитектура:

- База знаний: факты и правила «если-то»
- Механизм вывода: прямой/обратный вывод
- Интерфейс с пользователем

Ограничения:

- Требует явной формализации знаний
- Не обучается на данных
- Проблема приобретения знаний

Пример: MYCIN (диагностика инфекций, 1970-е)

Архитектура экспертной системы

Индуктивный подход: машинное обучение (6)

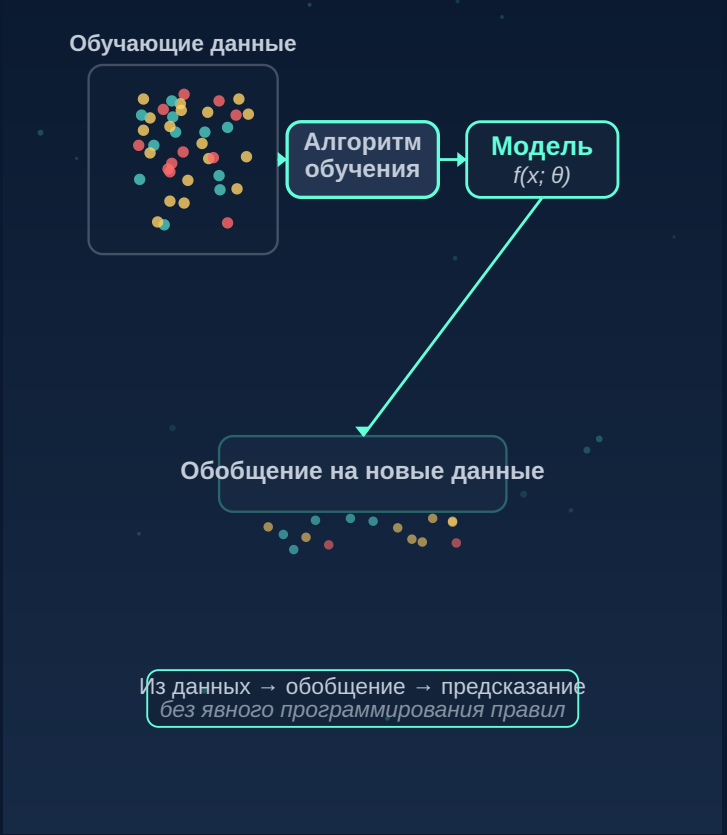
Машинное обучение — алгоритмы, обучающиеся на данных без явного программирования правил.

Фундаментальная идея: алгоритм автоматически извлекает закономерности из данных.

Преимущество перед дедуктивными системами: обобщение на новые данные без изменения архитектуры.

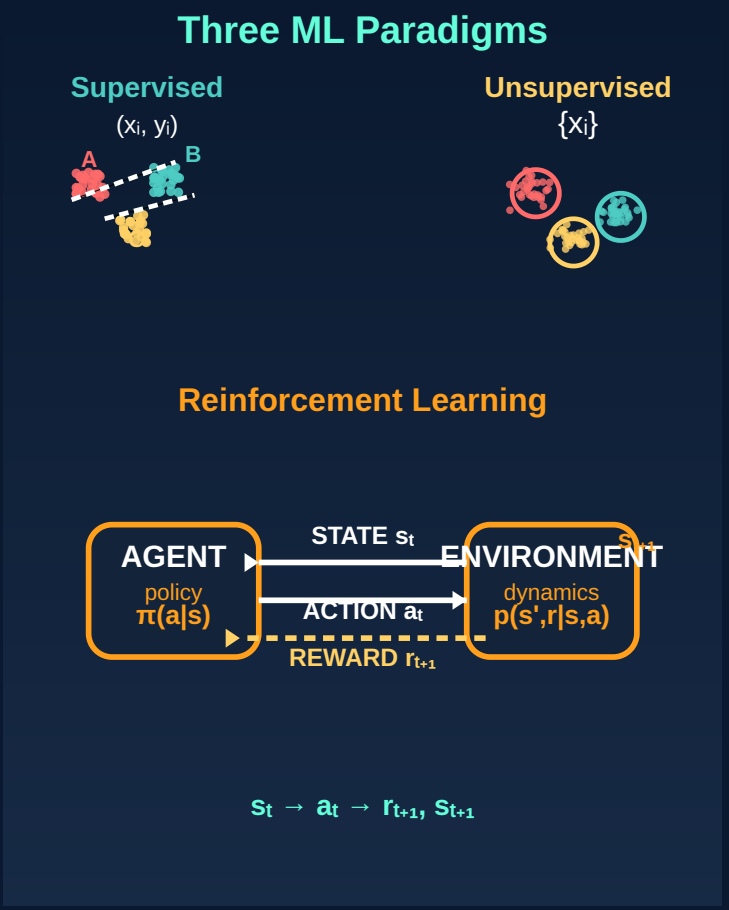
Пример: нейронные сети как универсальные аппроксиматоры.

Индуктивный подход: обучение на данны



Три парадигмы машинного обучения (7)

Обучение с учителем: пары (x_i, y_i) → предсказание
Обучение без учителя: только $\{x_i\}$ → структура
Обучение с подкреплением: агент ↔ среда → максимизация награды



Обучение с учителем: регрессия и классификация (8)

Регрессия (непрерывный выход)
- Функция потерь: MSE
- Пример: предсказание цен

Классификация (дискретный выход)
- Функция потерь: кросс-энтропия
- Пример: распознавание цифр

Единая формулировка:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^{N} L(y_i, f(\mathbf{x}_i)) + \lambda \Omega(f)$$

Обучение без учителя: кластеризация (9)

Кластеризация — группировка объектов по сходству.

Метрики качества:

- Силуэт: $s(i) = \frac{b(i)-a(i)}{\max\{a(i),b(i)\}}$
- Внутрикластерная дисперсия

Алгоритмы:

- K-means: минимизация внутрикластерной дисперсии
- Иерархическая: построение дендрограммы
- DBSCAN: кластеры произвольной формы

Применения: сегментация рынка, обнаружение аномалий.

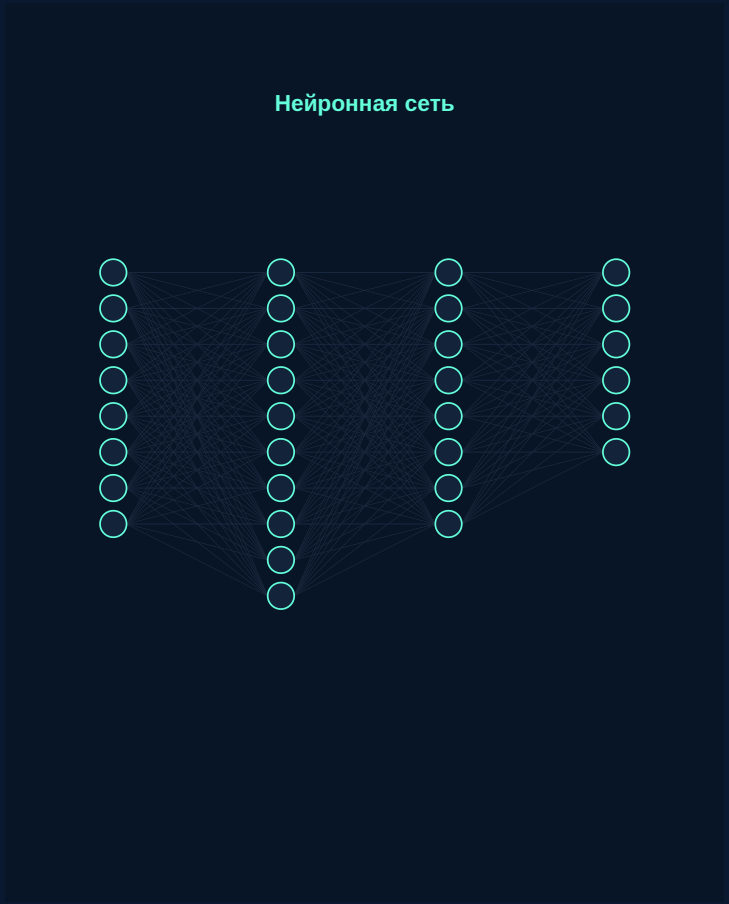
Нейронные сети: математическая модель (10)

Нейрон — вычислительная единица:

$$a = \sigma(\mathbf{w}^T\mathbf{x} + b)$$

Функции активации:
- Сигмоида: $\sigma(x) = \frac{1}{1+e^{-x}}$
- ReLU: $\sigma(x) = \max(0, x)$
- Tanh: $\sigma(x) = \tanh(x)$

Архитектура: входной слой → скрытые слои → выходной слой



Теоремы универсальной аппроксимации (11)

Теорема Колмогорова-Арнольда (1957):
Любая непрерывная функция представима как суперпозиция функций одной переменной:

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

Теорема Цыбенко (1989):
Сеть с одним скрытым слоем и сигмоидой аппроксимирует любую непрерывную функцию:

$$\forall f \in C(I_n), \forall \epsilon > 0, \exists F(\mathbf{x}) = \sum_{j=1}^{N} \alpha_j \sigma(\mathbf{w}_j^T \mathbf{x} + b_j) :$$

$$\sup_{\mathbf{x} \in I_n} |F(\mathbf{x}) - f(\mathbf{x})| < \epsilon$$

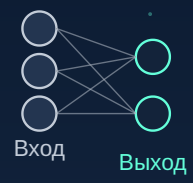
Ограничение: число нейронов может быть экспоненциально большим.

Сравнение: однослойная vs многослойная сеть (12)

Однослойная: $N_{shallow} \sim O(2^n)$ — экспоненциальный рост
Многослойная: $N_{deep} \sim O(n \cdot k)$ — линейный рост

Сравнение архитектур нейронных сетей

Однослойная сеть
(Shallow Network)



Сложность: O(2ⁿ)
экспоненциальный рост

Многослойная сеть
(Deep Network)

Сложность: O(n·k)
линейный рост

Глубокие сети эффективнее для сложных функций
иерархическое извлечение признаков с меньшими затратами

Преимущество глубины (13)

Экспоненциальная сложность плоских сетей:
Для представления иерархических функций требуется экспоненциальное число нейронов.

$$N_{shallow} \sim O(2^n)$$

Линейная сложность глубоких сетей:
Та же задача решается с полиномиальным числом параметров.

$$N_{deep} \sim O(n \cdot k)$$

Примеры:
- ResNet-152: 152 слоя (ImageNet)
- BERT: 12-24 слоя (NLP)
- GPT-3: 96 слоёв, 175B параметров

Глубина решает: экспонента vs полином

Плоские сети
Экспоненциальная сложность:
Nhhalxhnə ~ O(2ⁿ)

Глубокие сети
Линейная сложность:
Neeem ~ O(n·k)
n — размерность входа, k — глубина

Примеры глубоких архитектур:

ResNet-152	BERT	GPT-3
152 слоя	12-24 слоя	96 слоёв
Классификация ImageNet	*Обработка языка*	*175 млрд параметров*

Глубина позволяет выделять иерархические признаки с меньшей вычислительной стоимостью

Теорема Эльдана-Шамира о глубине (14)

Теорема (Eldan & Shamir, 2016):
Существуют функции, которые 3-слойная сеть представляет точно, а 2-слойная требует экспоненциального числа нейронов.

$$\exists \text{ 3-layer network} : N_3 = O(d)$$

$$\forall \text{ 2-layer network} : N_2 = \Omega(e^d)$$

Интуиция: глубокая сеть эффективно представляет композицию функций

$$f = f_L \circ f_{L-1} \circ \cdots \circ f_1$$

Практическое значение:
- Объясняет эффективность глубоких архитектур
- Глубина ≠ больше параметров, а качественное изменение выразительности

Теорема Эльдана-Шамира о глубине (2016

Существуют функции, которые глубокая сеть с 3 слоями может представить точно, а сеть с 2 слоями требует экспоненциального ч нейронов.

Математическая формулировка:

Существует распределение p(x) и функция $f : R^d \to R$ такая

∃ 3-layer network : $N_3 = O(d)$

∀ 2-layer network : $N_2 = \Omega(e^d)$

Визуальное сравнение:

2 слоя: $N_2 = \Omega(e^d)$

3 слоя: $N_3 = O(d)$

Интуиция:

Глубокие сети эффективно представляют композицию функ

$f = f_L \circ f_{L-1} \circ \cdots \circ f_1$

Практическое значение:
- Объясняет эффективность глубоких архитектур в реальны
- Глубина ≠ просто больше параметров, а качественное изм
- Основа для проектирования архитектур ResNet, DenseNet

Градиентный спуск (15)

Градиентный спуск — итеративная минимизация функции потерь:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$$

Компоненты:
- θ — параметры модели
- η — скорость обучения
- $\nabla_\theta J$ — градиент функции потерь

Варианты:
- SGD: стохастический градиентный спуск
- Momentum: учёт инерции
- Adam: адаптивная скорость обучения

Автоэнкодеры: сжатие без учителя (16)

Автоэнкодер — архитектура для нелинейного снижения размерности:
Энкодер: $\mathbf{z} = f_\phi(\mathbf{x}) \in \mathbb{R}^d,\ d \ll D$
Декодер: $\hat{\mathbf{x}} = g_\theta(\mathbf{z})$

Применения: сжатие, денойзинг, извлечение признаков.

Автоэнкодер

Вариационный автоэнкодер (VAE) (17)

VAE — вероятностная версия автоэнкодера:
ELBO (Evidence Lower Bound):

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \mathrm{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$$

Ключевое отличие:
- AE: детерминированная точка в латентном пространстве

Преимущество: плавная интерполяция между объектами, генерация новых данных.

AE vs VAE: детерминированное vs вероятност

ELBO: log p(x) ≥ 𝔼q[log p(x|z)] - KL(q(z|x) || p(z))

Ключевое отличие:
AE: детерминированное отображение x → z → x̂
VAE: вероятностное отображение x → q(z|x) → z ~ q → x̂

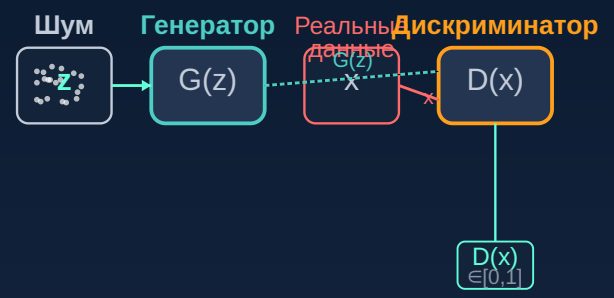
Генеративно-состязательные сети (GAN) (18)

GAN — минимаксная игра двух сетей:
Генератор: $G_\theta(\mathbf{z})$ создаёт фальшивые данные
Дискриминатор: $D_\phi(\mathbf{x})$ отличает реальные от фальшивых
Целевая функция:

$$\min_\theta \max_\phi \mathbb{E}_{\mathbf{x}\sim p_{data}}[\log D_\phi(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D_\phi(G_\theta(\mathbf{z})))]$$

Равновесие: $D_\phi(\mathbf{x}) = 0.5$ (генератор идеален)

Генеративно-состязательная сеть (GAN)



min max V(D,G) = 𝔼x[log D(x)] + 𝔼z[log(1 - D(G(z)))]
G D

Равновесие Нэша:
D(x) = 0.5 для всех x
(генератор создает неотличимые данные)

Диффузионные модели (19)

Прямой процесс: постепенное добавление шума

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

Обратный процесс (обучаемый): постепенное удаление шума

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t))$$

Ключевая формула:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$$

Преимущества: отсутствие коллапса мод, высокое качество.

Диффузионные модели

Прямой процесс: $x_0 \to x_1 \to \ldots \to x_t \to \ldots \to x_T$

$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{(1-\beta_t)}x_{t-1}, \beta_t I)$

Обратный процесс: $x_T \to \ldots \to x_t \to \ldots \to x_1 \to x_0$

$p\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu\theta(x_t,t), \Sigma\theta(x_t,t))$

$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{(1-\bar{\alpha}_t)}\, \varepsilon,\ \varepsilon \sim \mathcal{N}(0, I)$

Поток генерации в диффузионных моделях (20)

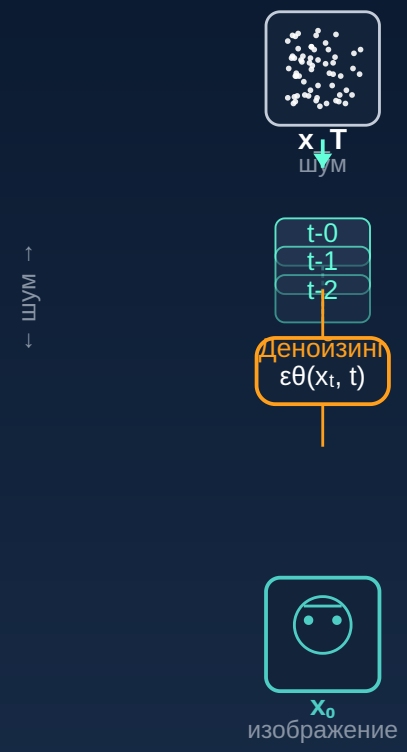
Процесс генерации:

1. **Старт:** $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ (чистый шум)
2. **Итерация:** модель предсказывает шум $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$
3. **Денойзинг:**

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$$

4. **Результат:** $\mathbf{x}_0$ — фотореалистичное изображение

Диффузионная генерация



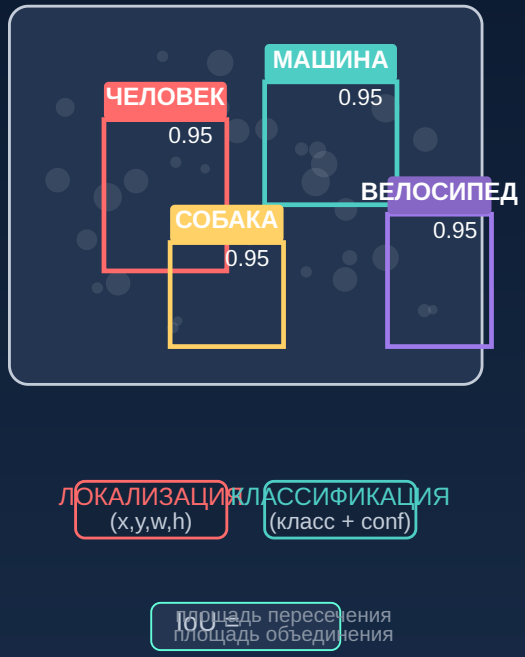
Компьютерное зрение: детекция объектов (21)

Детекция объектов = локализация + классификация:
Локализация: $\mathbf{b} = (x_{min}, y_{min}, x_{max}, y_{max})$
Классификация: $c \in \{1, \ldots, C\}$
Метрика IoU (Intersection over Union):

$$\mathrm{IoU} = \frac{|\mathbf{b}_{pred} \cap \mathbf{b}_{gt}|}{|\mathbf{b}_{pred} \cup \mathbf{b}_{gt}|}$$

Архитектуры:
- Двухступенчатые: Faster R-CNN (точность)
- Одноступенчатые: YOLO (скорость)
- Трансформерные: DETR

Детекция объектов



YOLO: You Only Look Once (22)

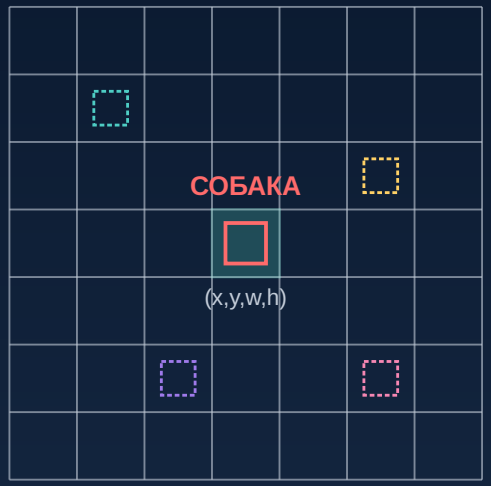
Принцип: разделение изображения на сетку $S \times S$, каждая ячейка предсказывает:
- B боксов с координатами (x, y, w, h) и уверенностью

Функция потерь:

$$\mathcal{L} = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} 1_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{}$$

Преимущества: реальное время, глобальный контекст.

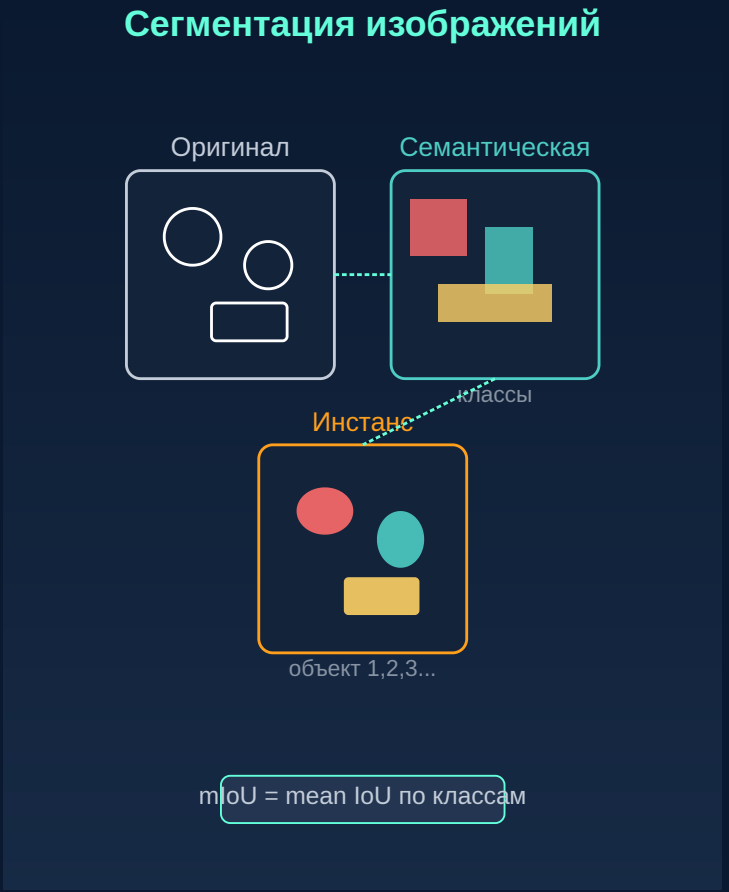
YOLO: You Only Look Once



Каждая ячейка предсказывает:
- B bounding boxes (x,y,w,h, confidence)
- Условные вероятности классов P(Class|Object)
- Итог: confidence × P(Class|Object)

Сегментация изображений (23)

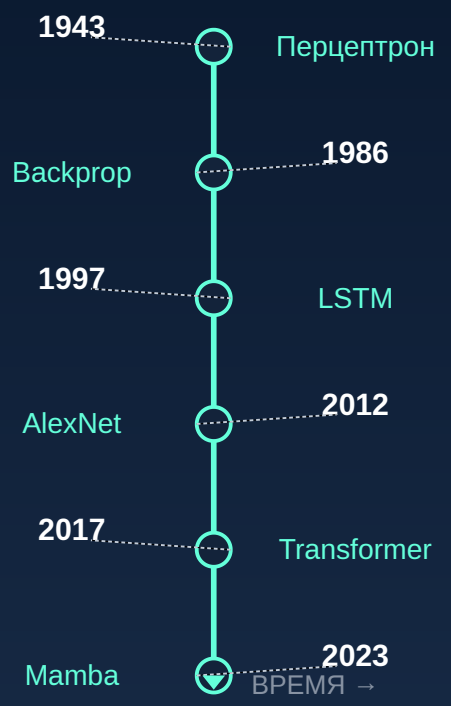
Семантическая сегментация — каждый пиксель → класс (FCN, U-Net)
Сегментация инстансов — каждый объект → уникальная маска (Mask R-CNN)
Метрика: mean IoU
Применения: автономные автомобили, медицинская визуализация.



Хронология развития архитектур (24)

Год	Архитектура	Ключевой вклад	Сложность
1943	Перцептрон	Модель нейрона	Линейная
1986	Backprop	Обучение многослойных сетей	-
1997	LSTM	Долгосрочная память	$O(N)$
2012	AlexNet	Глубокие свёрточные сети	$O(N)$
2017	Трансформер	Само-внимание	$O(N^2)$
2023	Mamba	Выборочное состояние	$O(N)$

Эволюция архитектур



Трансформер: архитектура (25)

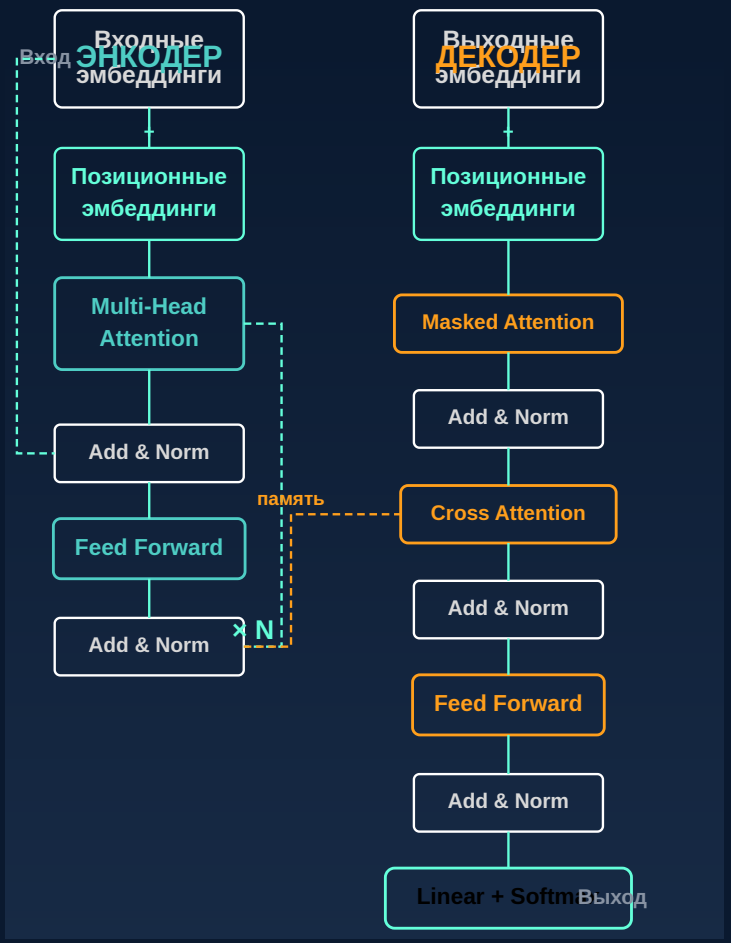
Энкодер-декодер с механизмом внимания:
Внимание (Attention):

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

Много-головое внимание (Multi-Head):

$$\mathrm{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\mathbf{W}^O$$

Преимущество: параллельная обработка последовательности.
Недостаток: квадратичная сложность $O(N^2)$.



Mamba: линейная сложность (26)

Проблема трансформеров: $O(N^2)$ не масштабируется на длинные последовательности.
Решение Mamba: выборочное пространство состояний (Selective SSM):

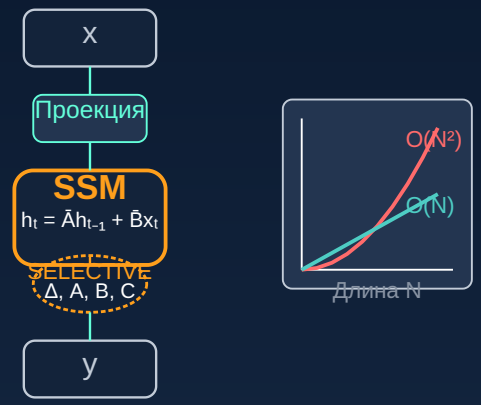
$$\mathbf{h}_t = \bar{\mathbf{A}}(\Delta_t)\mathbf{h}_{t-1} + \bar{\mathbf{B}}(\Delta_t)\mathbf{x}_t$$

$$\mathbf{y}_t = \bar{\mathbf{C}}(\Delta_t)\mathbf{h}_t + \bar{\mathbf{D}}(\Delta_t)\mathbf{x}_t$$

Ключевые свойства:
- **Адаптивная память:** динамическое решение о сохранении информации
- **Игнорирование шума:** без дополнительных вычислений
- **Линейная сложность:** $O(N)$

Результат: обработка контекстов 100K+ токенов.

Mamba: Selective SSM



KAN: Kolmogorov-Arnold Networks (27)

Альтернатива MLP на основе теоремы Колмогорова-Арнольда:
Теорема: любая непрерывная функция представима как:

$$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^{n} \phi_{q,p}(x_p) \right)$$

Архитектурное отличие:
- MLP: обучаемые веса + фиксированные нелинейности
- KAN: фиксированные веса + обучаемые функции на рёбрах

Преимущества:
- Интерпретируемость (каждая функция ϕ_{ij} соответствует влиянию признака)
- Меньше параметров при той же выразительности

KAN

Kolmogorov-Arnold Networks

$$f(x_1,\ldots,x_n) = \Sigma\, \Phi_i(\, \Sigma\, \varphi_{ij}(x_j)\,)$$

MLP:
веса w_{ij}

KAN:
функции $\varphi_{ij}(x)$

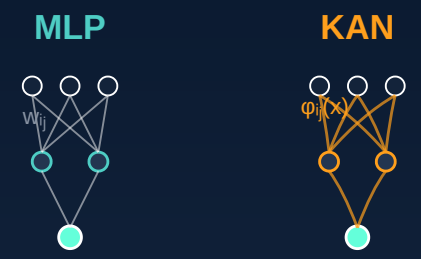
Сравнение MLP и KAN (28)

Характеристика	MLP	KAN
Что обучается	Веса w_{ij}	Функции $\phi_{ij}(x)$
Нелинейность	Фиксирована (ReLU, sigmoid)	Адаптивна
Интерпретируемость	Низкая	Высокая
Параметры	Много	Мало
Теоретическая основа	Теорема Цыбенко	Теорема Колмогорова-Арнольда

Формула KAN-слоя:

$$\mathbf{y}_j = \sum_{i=1}^{n} \phi_{j,i}(x_i)$$

MLP vs KAN



MLP: $y = \sigma(W_2 \cdot \sigma(W_1 x + b_1) + b_2)$

KAN: $y = \Sigma\, \varphi_i(\, \Sigma\, \varphi_{ij}(x_j)\,)$

Будущее архитектур ИИ (29)

Четыре ключевых направления:

1. **Вычислительная эффективность**
- $O(N^2) \to O(N)$ (Mamba, SSM)
- Edge AI, квантование
2. **Интерпретируемость**
- От «чёрных ящиков» к прозрачным моделям (KAN)
- Визуализация внимания, формальная верификация
3. **Мультимодальность**
- Единые архитектуры для текста, изображений, аудио, видео
- Кросс-модальное рассуждение
4. **Этическая ответственность**
- Снижение предвзятости
- Прозрачность и подотчётность

Резюме: ключевые выводы (30)

1. **Два фундаментальных подхода к ИИ:**
- Дедуктивный (экспертные системы): знания → вывод
- Индуктивный (нейросети): данные → обобщение
2. **Глубина имеет значение:**
- Теорема Эльдана-Шамира: 3 слоя эффективнее 2
- Экспоненциальная сложность плоских сетей
3. **Эволюция архитектур:**
- Перцептрон → LSTM → Трансформер → Mamba
- Тренд: линейная сложность $O(N)$
4. **Будущее:**
- Эффективность + интерпретируемость + мультимодальность + этика